



МОСКОВСКИЙ УНИВЕРСИТЕТ ИМЕНИ С.Ю. ВИТТЕ

Факультет Информационных технологий

Кафедра Кафедра информационных систем

Направление подготовки/Специальность Прикладная информатика

ОТЧЕТ

о прохождении

Производственная практика

/вид практики/

Эксплуатационная практика

/тип практики/

Студента Дронова Яна Витальевича

(фамилия, имя, отчество)

Место прохождения практики ЧОУВО «МУ им. С.Ю. Витте»

Период прохождения практики с 20.04.2026 г. по 31.05.2026 г.

г. Москва 2026 г.

Оглавление

Введение.....	4
Цели практики	5
Задачи практики	5
1. Анализ предметной области.....	7
1.1 Теоретические основы методов предобработки изображений	7
1.1.1 Как компьютер видит изображение? Свёрточные нейронные сети	9
1.2 Как происходит считывание и обработка картинок (OpenCV, PIL, torchvision)	11
1.3 Основные архитектуры нейросетей для анализа изображений	12
1.4. Метрики качества для задачи обнаружения объектов	13
2. Использование методов предобработки с помощью нейросетей (библиотеки: Alumentations, PyTorch, Ultralytics)	14
2.1. Зачем нужна предобработка.....	14
2.2. Основные методы предобработки.....	15
2.3. Библиотеки для предобработки	16
3. Формирование датасета для обучения.....	16
3.1 Источники данных и критерии поиска	17
3.2 Сбор и фильтрация изображений, разметка данных, Golden Dataset.....	18
4. Реализация и результаты	18
4.1 Выбор архитектуры и инструментов	19
4.1.1. Модель для выполнения задания	19
4.1.2. Визуальная оболочка и отказ от начальной идеи	19
4.2 Дообучение модели, обработки изображений и механизмы.....	20
4.2.1 Первая попытка: OBB-модель	20
4.2.2 Вторая попытка: VisDrone-модель, дообучение	20
4.2.3 Тайлинг (разбиение на части)	21
4.3. Реализация структуры программы.....	22
4.4 Используемые метрики.....	23
4.6 Верификация на Golden Dataset.....	25

4.7 Проблемные распознавания	25
Заключение.....	30
Список литературы	32

Введение

С началом эксплуатационной практики у нас на выбор было несколько типов задач, самые популярные типы это работа с датасетом из документов, преимущественно имеющих текст и работа с датасетом из изображений. Я выбрал взаимодействие с изображениями.

Суть моей задачи заключается в следующем: у меня есть датасет из 100 фотографий местности с высоты дрона и открытый датасет VisDrone из 6470 фотографий, также я имею код, в котором реализован детектинг типов объектов (то есть обученная модель может отличить грузовую машину от легковой и обнаружить её). На словах алгоритм звучит довольно просто, но на деле нам нужно разобраться в многих деталях, связанных с машинным обучением, например нужно понять что такое Computer Vision (область, в которой будет проходить моя работа), что такое предобработка данных, слои, ядра, и многое многое другое.

Я считаю, что моя задача актуальная, детектинг объектов с высоты полета дрона используется чаще всего либо в военных ситуациях, либо в волонтерских поисках людей по местности. Также можно внедрить автоматизацию при поиске преступника, либо его автомобиля. Главная проблема при работе с аэрофотоснимками – это огромный объем данных. Один разведывательный полёт на дроне дает тысячи снимков, которые чаще всего просматривают вручную, это занимает часы. При этом ещё и работает человеческий фактор и усталость, которые значительно уменьшают эффективность поиска на фотографии, человек нечаянно может пропустить нужное. Самый яркий пример - поисково-спасательный отряд «Лиза Алерт», раньше они вручную изучали сфотографированную местность, теперь же в рамках проекта New «Detection Model for Liza Alert» туда внедряется новая модель YOLO11n, точность обнаружения 97,6%, у модели

2,7 млн параметров (у YOLOv5s 7 млн), то есть это самое эффективное и быстрое решение проблемы и естественно нужное в наших реалиях.

Цели практики

Осознав написанное выше, становится ясно, что главных целей в этой практике у меня две: первая заключается в непосредственном выполнении задачи, а во второй мне нужно изучить выполнение этой задачи логически, нужно понять, что делается, какими способами, как можно сделать выполнение задачи ещё лучше.

Также целью является изучение способов обучения моделей и использования их на реальных изображениях.

Задачи практики

Я выделил такие задачи по мере выполнения практической работы:

- 1) Ознакомиться с условием задачи и решить, как ее выполнять.
- 2) Изучить базовые понятия Computer Vision, разобраться в устройстве сверточных нейронных сетей (CNN) и их применении для анализа изображений, изучить современные архитектуры детекторов объектов (YOLO приоритет), освоить метрики оценки качества моделей.
- 3) Сформировать датасет из изображений (найти в интернете готовый, либо набрать самостоятельно), выполнить предобработку изображений, выбрать лучший инструмент для работы с изображениями, сформировать golden датасет (скорее всего он даже и не понадобится в явном виде).
- 4) Дообучить модель на датасете (как раз YOLO модель), оценить качество модели на датасете и golden датасете, визуализировать результаты.

5) Провести сравнительный анализ с моим коллегой, выполняющим схожее задание (будет произведен обмен данными, также обсуждены подходы к обработке и обучению модели).

6) Оформить GitHub репозиторий со всеми датасетами, моделью и инструкцией, как всем этим пользоваться.

7) Создать данный отчёт.

1. Анализ предметной области

1.1 Теоретические основы методов предобработки изображений

Для начала нам стоит разобраться в том, что такое Computer Vision и почему мы на данный момент находимся в этом направлении при решении задачи.

Computer Vision – это область искусственного интеллекта, которая учит компьютеры видеть и обрабатывать визуальную информацию также, как это работает у человека. Vision дословно переводится как зрение. Но дело в том, что одного зрения недостаточно. Наш мозг помогает глазам мгновенно обработать информацию и понять, что находится перед нашим взором. Компьютер же так не может, он видит лишь огромное число пикселей, каждый из которых подразумевает в себе три значения, которые обозначают яркость пикселя. Пиксель это по сути независимая мельчайшая точка на изображении и компьютер никак не может логически сопоставить эту точку с соседней.

Мы используем Computer Vision в тех случаях, когда человеческий труд может расцениваться как неэффективный (зачем ставить людей как конвейер на анализ снимков по 8 часов, если это может сделать компьютерная модель за более кратчайший срок, причем безошибочно), либо же когда какие-то процессы можно легко автоматизировать. Работа, которую я выполняю, легко поддается автоматизации и нуждается в ней, пример я уже описывал выше. Актуальность использования CV увеличивается с каждым годом, потому что, во-первых, время не стоит на месте и мы сейчас проживаем этап популяризации и массового внедрения искусственного интеллекта, а во-вторых, множество процессов нуждается в автоматизации, ведь можно потратить сохранённое время на более сложные

и важные вещи, одной моделью можно заменить несколько десятков человек.

Прежде чем начать работу, нужно осознать главные понятия, которые я буду встречать, например объект. Вообще, моя конкретная задача – это детекция объектов, это в свою очередь задача компьютерного зрения, то есть я касаюсь определенного сегмента CV. Детекция объектов – это поиск и локализация отдельных объектов на фото или видео. Процесс такой: сначала модель анализирует изображение и определяет, какие объекты она нашла на фото и к каким классам эти объекты относятся. Далее каждому объекту присуждается прямоугольная рамка, которая показывает границы объекта и указывает его местоположение на кадре; соответственно рамка отрисовывается с помощью x и y минимальных и максимальных.

Есть похожая на детекцию задача, а именно классификация и сегментация, и я сейчас объясню, почему выбрана именно детекция.

Классификация – это задача компьютерного зрения, но она заключается в том, чтобы просто определить, к каким классам принадлежат объекты, это недостаточно для определения местоположения. Сегментация идет чуть глубже, чем детекция – модель не просто обводит местоположение найденных объектов, она еще и определяет идеально точные границы на уровне пикселей. При выполнении моей задачи я буду придерживаться задачи детекции, потому что я считаю не особо нужным выделять идеальные границы автомобилей, прямоугольника будет достаточно. Этот прямоугольник, кстати, называется Bounding Box, в машинном обучении это базовый метод разметки на изображениях.

Предобработка изображения – это приведение изображения в удобный вид, с которым и будет работать наша модель. Чаще всего при

предобработке изменяют размер (важная деталь, об этом я ещё вспомню), нормализация яркости, устранение шумов, разбиение на тайлы (плитки).

Есть такой термин как дообучение, мы берем уже готовую модель и обучаем дополнительно на своих фотоснимках, чтобы она была более точной.

Валидация – после обучения проверка модели на фотографиях, которые она не видела ранее.

Golden Dataset – это набор фотографий, где я лично заранее проверил все объекты, нужно для финальной оценки качества.

Ultralytics – библиотека, через которую я загружаю мою модель.

1.1.1 Как компьютер видит изображение? Свёрточные нейронные сети

Для нас фотография это просто картинка. Для компьютера это огромное количество чисел. Если взять фотографию размером 1920x1080, то если мы подадим это через нейросеть, то получится $1920 * 1080 * 3$ входных весов на первом слое (3 это каналы RGB). Число большое, вычислительные затраты будут огромными, да и модель запомнит шумы. Для решения этой проблемы были введены свёрточные нейронные сети (CNN).

Свёртка – это операция, при которой маленькое ядро скользит по изображению и вычисляет сумму поэлементных произведений. Ядро - маленькая матрица весов (3x3, 5x5, 7x7), каждое ядро распознает один тип признаков: ядро вертикальных границ ищет резкие переходы яркости по горизонтали, горизонтальные по вертикали, а ядро углов смотрит это по

границам. В современной CNN ядра не задаются вручную, а обучаются на примерах.

Допустим, у нас есть ядро 3×3 , мы используем 16 разных ядер, шаг 1 пиксель. По итогу 16 ядер одновременно скользят по изображению, каждое дает выходную карту признаков. Еще часто используют pooling – это сжатие, оно уменьшает размерность, оставляя самые важные значения.

В CNN слои расположены иерархически. Чем глубже слой, тем более сложные признаки он ищет. Например, первый слой может искать края, границы между объектом и остальным, второй слой ищет простые формы, дуги, углы объекта. Третий слой ищет части объектов, например колесо, окно, фару. Далее четвертый слой ищет объект целиком, если весь объект есть на фото, по итогу выходные слои дают координаты коробки с объектом, класс объекта и уверенность в этом объекте (уверенность в объекте это мера того, насколько алгоритм уверен в распознавании объекта на изображении, считается от 0 до 1).

Я увидел в интернете интересную трактовку примера сжатия информации в CNN, это очень наглядно:

Представьте, что на вход подаётся аэроснимок парковки с 50 машинами.

На входе (слой 0): $640 \times 640 \times 3 \sim 1.2$ млн чисел - каждый пиксель со своим цветом.

После слоя 1: $320 \times 320 \times 32 \sim 3.2$ млн чисел - уже не пиксели, а карты границ.

После слоя 2: $160 \times 160 \times 64 \sim 1.6$ млн чисел - карты простых форм.

После слоя 3: $80 \times 80 \times 128 \sim 0.8$ млн чисел - карты частей машин.

После слоя 4: $40*40*256 \sim 0.4$ млн чисел - карты целых машин.

На выходе: список из (координаты рамки + класс + уверенность) - $\sim 50*(4+1+1) = 300$ чисел.

Сжатие в 4000 раз! При этом важная информация (где находятся машины) сохранилась, а шумы и неважные детали отфильтровались. Это колоссально уменьшает количество и скорость вычислений, также не допускает переобучения.

При выполнении своей работы я буду использовать модель YOLO, а она в свою очередь является одной из самых известных архитектур свёрточных нейронных сетей для задач компьютерного зрения. Главная особенность YOLO заключается в том, что она объединяет поиск и классификацию объектов в одну задачу, выполняя всё за один проход (потому и такое название You Only Look Once). Снимки с дрона это непростая задача для CNN, например машина на снимке с высоты занимает всего 20-100 пикселей, ее сложно распознать, я столкнусь с путаницей классов, также могут мешать тени, кусты, трещины в асфальте, также разный масштаб снимков. Но об этом я расскажу в пункте с реализацией.

1.2 Как происходит считывание и обработка картинок (OpenCV, PIL, torchvision)

Просто взять и скинуть модели фотографию со словами «детекть» не получится. Модель просто не сможет прочитать фото. При считывании фотографии с диска её нужно преобразовать в формат, который модель может считать, а после детекции отрисовать результаты и вернуть отправителю. Python готов помочь с этим, и среди библиотек можно найти именно те, которые упростят нам задачу.

Первая библиотека – Pillow, она необходима для базовой работы с изображениями, она проста, интегрируется с tkinter, но медленная. В своем проекте я буду использовать её для открытия изображения, создании объекта для рисования, рисования прямоугольников, текста, и конвертера в tkinter.

Вторая библиотека – OpenCV, она создана для задач компьютерного зрения и используется для обработки изображений. В проекте я могу считывать изображение в BGR, конвертировать в RGB, изменять размер фото, рисовать рамки, текст, преобразовывать для показа в окне, делить картинки на тайлы, масштабировать при зуме.

Есть ещё третья библиотека – TorchVision, но она мне не пригодится, так как аугментации YOLO делает сам (аугментация это увеличение количества данных путем трансформаций исходных изображений), тем более TorchVision не поддерживает OBB (OBB мне нужно будет для отрисовки не только горизонтальных коробок).

1.3 Основные архитектуры нейросетей для анализа изображений

В какой сфере мы работаем - поняли, поняли и то, с какими конкретно библиотеками я буду работать, осталось понять, почему мною была выбрана архитектура YOLO, помимо того, что она самая популярная.

Детекторы делятся на двухступенчатые, одноступенчатые и трансформерные. Сначала были двухступенчатые детекторы и были они очень медленные. Принцип работы был таков: сначала предлагались регионы с объектами, затем классифицированы каждый регион. Происходило два цикла, что было уже довольно долгим процессом. Люди задумались: а зачем сначала искать регионы, потом классифицировать, если

можно делать это одновременно? Так, в 2016 году появилась YOLO, своим названием объясняющая одноступенчатость. Механизмы такого типа стали значительно ускорять работу и YOLO стали постоянно модифицировать. С первой версии создатели обновились до 11, она была выпущена в 2024 году. Есть еще похожий одноступенчатый детектор SSD, он быстрее на мелких объектах, но YOLO лучше понимает глобальный контекст, популярнее и развивается активнее. Трансформерные детекторы стали набирать популярность в 2020 году после выхода DETR. Это мощная структура, трансформер лучше понимает сцену целиком - например, может использовать пустую парковку как контекст, чтобы не детектировать машину там, где их быть не может, но она медленно обучается, плохо работает с мелкими объектами и требует огромных датасетов, поэтому для моей работы такое не подходит.

1.4. Метрики качества для задачи обнаружения объектов

Теперь стало ясно, какую архитектуру я буду использовать, осталось разобраться в последнем: а как оценить работу модели? Как понять, что моя модель отлично всё нашла, либо же плохо, не смогу же я написать это текстом. В задаче обнаружения объектов используется набор метрик, которые позволяют объективно сравнить разные модели и настройки.

Нужно понять, что является правильным для модели, для этого обычно используют IoU – метрику совпадения рамок. Считается от 0 до 1, 1 это идеальное совпадение. В задачах детекции стандартный порог $\text{IoU} \geq 0.5$.

Для класса объектов в задаче детекции можно применить матрицу ошибок и в моей задаче можно использовать её так: True Positive(нашли машину), False Positive(нашли машину там, где её нет), True

Negative(определили, что машины тут нет), False Negative(машина есть, но ее не нашли).

Precision – точность, количество настоящих машин, делённое на количество объектов, определённых машинами. $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$

Recall – полнота, из всех настоящих машин на фото, какую долю модель успешно обнаружила. $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$

AP – средняя точность, mAP – основная метрика детекторов, среднее AP по всем классам.

2. Использование методов предобработки с помощью нейросетей (библиотеки: Albumentations, PyTorch, Ultralytics)

2.1. Зачем нужна предобработка

Предобработка изображений это важный этап в задаче компьютерного зрения. От того, насколько качественно подготовлены данные, зависит то, сможет ли модель обучиться и работать на реальных изображениях, поэтому, стоит немного подробнее разобраться в том, что при предобработке используют и как.

Одна из самых популярных проблем заключается в том, что дроны снимают кадры в разном качестве, какие-то из них оснащены камерой в 1080p, дроны послабее 720p, а более навороченные имеют чудесные 4K. Размеры фотографий сильно различаются по пикселям и при предобработке данных производится ресайз фотографий до фиксированного размера. Разные фотографии имеют разное освещение, что тоже мешает при анализе фото, поэтому при предобработке данных чаще всего яркость нормализуют или/и корректируют контраст. Сглаживают шум на фото, нормализуют цветные каналы.

При изменении размера фото до размера модели машины (в моем примере) становятся слишком шакальными для детекции, поэтому исходное фото при предобработке разбивается на тайлы (куски), каждый из которых проходит детекцию в более хорошем качестве. Также к предобработке данных относится аугментация (уже писал про неё, но напомним: это искусственное создание примеров с помощью уже существующих).

Теперь выделю более важные методы, объясняя, как они используются в моём проекте.

2.2. Основные методы предобработки

К основным методам предобработки относится нормализация. Это приведение значений пикселей к стандартному диапазону (от 0.0 до 1.0). Так как пиксели обычно имеют значения от 0 до 255, то нормализация вычисляется $norm = image / 255.0$. В моём проекте нормализация автоматически выполняется внутри YOLO.

Ещё один метод – ресайз. В YOLO стандартный размер 640x640 px. При ресайзе искажаются пропорции объектов, но YOLO помогает и в этом: справляется с этим за счёт обучения на разных масштабах.

Тайлинг - самый важный для меня этап, заключается он в разбиении большого изображения на перекрывающиеся куски. Это очень важно, потому что при сжатии идентифицировать мелкие машинки становится практически невозможно и я с этим столкнулся при создании программы. Добавил разбиение на тайлы и с более лучшими пикселями модель стала идентифицировать машины. Вместе с тайлингом используется перекрытие (overlap). Зачем оно нужно: при делении изображения на тайлы машины может идеально оказаться в месте разреза, и тогда идентифицировать

модель не сможет либо засчитать машину дважды, перекрытие гарантирует, что машина попадёт хотя бы в один тайл.

Есть также аугментация, но и она автоматически используется в YOLO, например случайное изменение яркости от -0.4 до +0.4, изменение насыщенности, тона, масштабирование, поворот и прочее.

2.3. Библиотеки для предобработки

Существует несколько популярных библиотек для предобработки изображений в CV. Например, Albumentations – библиотека для аугментаций, оптимизированная для скорости и разнообразия. Достаточно импортировать и указать параметры в переменной. Мне это не нужно, потому что YOLO уже имеет встроенные методы. PyTorch библиотека предоставляет модуль torchvision.transforms для предобработки и аугментации. Это стандартный инструмент для экосистемы PyTorch. Но мне оно опять же не нужно, но из-за того, что YOLO напрямую работает с numpy-массивами и всё делает сам. Ultralytics это библиотека(и целая экосистема), которая предоставляет мне модель YOLO, она включает в себя все необходимые методы предобработки и аугментации через параметры тренировки.

3. Формирование датасета для обучения

Датасет – это коллекция данных, которая используется для обучения нейросетей, анализа информации, словно учебник. На этом учебнике модель учится находить закономерности, но опирается на них не как безусловные примеры, а как на схожести. Вручную собирать датасет это долго и мучительно, поэтому я решил реализовать это таким образом: я

вручную собираю 100 фотографий с дрона, на которых есть автомобили, вместе с этим я импортирую открытый датасет с 6400+ фото, он уже весь настроенный. Я имею ввиду то, что создатель датасета уже разметил машины и заполнил нужные столбцы. Добавил в программе для готового датасета кнопку, можно будет выбрать либо с моего датасета фото для детекции, либо программа вытащит одну случайную из готового датасета. Я ознакомился с другими работами, связанными с задачами компьютерного зрения и увидел, что многие коллеги используют открытые датасеты, они удобнее, уже собраны и размечены. В моём проекте используется модель YOLOv8n, которая обучена на датасете VisDrone. Ранее я пытался использовать датасет DOTA (Dataset for Object Detection in Aerial Images), он требовал другую модель(с поддержкой OBB), разделяла автомобили на классы по пикселям (а это очень неудобно, фотографии разного размера и легковые автомобили на низкий высоте будут считаться за грузовые), поэтому я решил остановиться на модели, обученной на VisDrone, детектирование шло так себе, поэтому было принято решение дообучить модель с помощью готового датасета VisDrone, что в итоге дало значительное улучшение качества распознавания.

3.1 Источники данных и критерии поиска

Для обучения модели я собрал 100 фотографий с высоты полёта дрона. Критерии отбора изображений были таковы: высота 30-100 метров, разная местность, освещение, разная плотность машин, разная погода и разный формат/разрешение фото. Также был взят открытый датасет VisDrone, созданный группой Тяньцзиньского университета (выбранный мною насчитывает 6470 фото). Датасет использован через платформу Activeloop Deep Lake и импортирован через библиотеку deeplake.

3.2 Сбор и фильтрация изображений, разметка данных, Golden Dataset

Собирать в открытом датасете ничего не нужно было, нужно было для моих 100 фотографий. Я собирал их, основываясь на том, что не стоит собирать изображения со слишком сильной размытостью, пересветом, и без дубликатов. В работе используется два формата разметки: txt и yolo. Txt это родной формат датасета, а yolo используется для ultralytics. Я не размечал VisDrone заново, так как датасет уже размечен. В программе я использую модель, дообученную на VisDrone с использованием разметки датасета. Насчёт Golden Dataset, мне он не пригодится, потому что это было бы бессмысленно по причине, что готовый открытый датасет VisDrone имеет разметку, подтвержденную сообществом. Раз он так долго висит на просторах интернета, значит все чудесно, если бы были проблемы с точностью разметки, пользователи бы сообщили об этом.

4. Реализация и результаты

После получения задания я начал строить у себя примерный план того, как реализовать данную программу. Логика была такова: нужно использовать модель, которая будет детектировать средства передвижения и различать их по классам. После классификации из текстовой базы данных я получаю информацию о том, какой дрон мне лучше использовать для съёмки и почему. Таким образом, моя программа подразумевает из себя мультимодальную RAG-систему.

4.1 Выбор архитектуры и инструментов

4.1.1. Модель для выполнения задания

Для начала я предположил, какая модель может мне подойти, остановился на YOLO, она самая популярная и эффективная. Выбрал YOLOv8 а не 11, потому что она более фундаментальная и выверенная, количество документации на неё огромное, сообщество составляет миллионы пользователей, на 11 версии пока что десятки тысяч. Также для датасета VisDrone существуют готовые веса именно под YOLOv8, для 11 версии я готовых весов не нашел, пришлось бы обучать модель с нуля. Разница в точности незначительна, но YOLOv8 лучше изучен и легче поддаётся отладке.

4.1.2. Визуальная оболочка и отказ от начальной идеи

Я сразу для себя решил, что мне нужна будет визуальная оболочка, я хочу, чтобы это было как целая программа с меню настроек и визуальным просмотром (поэтому большую часть моего кода занимает именно визуал, без визуала код был занял около 40 строчек). Вообще при старте выполнения задачи я не так для себя её интерпретировал и хотел искать не просто типы машин, но и марки. Сделал датасет и попробовал идентифицировать именно машины с марками. В процессе работы понял, что на фото с высоты дрона при ресайзе фото и существованием огромного количества марок авто это будет сделать почти невозможно, тем более готового правильно размеченного датасета под такую задачу я так и не смог найти, учить модель с нуля было бы тяжелым и долгим заданием, поэтому от этой задачи я отказался. При этом была одна интересная затея, сделать базу данных векторной, так процесс поиска марки был бы значительно ускорен, но всё упирается в сложность основного процесса, я подумал, что

не нужно усложнять себе задачу. Для визуализации был выбран tkinter и я хотел красиво оформить детекцию с кнопками и интерактивом.

4.2 Дообучение модели, обработки изображений и механизмы

4.2.1 Первая попытка: ОВВ-модель

При работе именно с детекцией возникали проблемы с идентификацией: на фото с 20 машинами находилась максимум одна, да и то с плохими процентами. Я не понимал, в чём дело, и, покопавшись в интернете, понял, что дело было в модели, которую я выбрал. Изначально мною была выбрана модель `yolov8n-obb.pt`, все потому, что я хотел отрисовывать bounding boxes не простым прямоугольником, а под углом, то есть он рисовался почти идеально по местоположению транспорта. Главная проблема заключается в том, что эта модель не обучена на фотографиях с высоты, соответственно будет выдавать плохие результаты. Чуть позднее я осознал ещё один минус: в obb модели классы транспорта делятся на маленький транспорт и большой транспорт, различаются они количеством пикселей. Так как я прикрепляю фотографии разного размера, при ресайзе модель по пикселям будет легчайшим образом считать грузовые автомобили как легковые и наоборот, поэтому от этой модели я благополучно отказался.

4.2.2 Вторая попытка: VisDrone-модель, дообучение

Выбор остановился на `yolov8n-visdrone.pt`, она популярна, для этой модели создано множество датасетов, но детекция работала всё также не очень. Для этого мне нужно было дообучить модель на аэрофотоснимках из готового датасета, сделано это было с помощью Google Collab с

использованием бесплатного GPU T4, по итогу получил модель best.pt, которую и использую в программе в качестве модели. Всё стало куда лучше: модель стала различать автомобили с коэффициентом уверенности 70-90%, для сравнения раньше он был 20-30% и то автомобили находились плохо. Сделал в коде настройку, чтобы можно было ползунком менять коэффициент уверенности и видеть, какие автомобили с каким процентом находятся моделью. Если поставить $\text{conf}=0.10$, то можно заметить, как модель выделяет части домов, либо дорожные знаки, траву, всё из-за того, что ресайз сжимает фото и модели становится сложнее распознать. Я придумал, как это улучшить.

4.2.3 Тайлинг (разбиение на части)

На этом моменте мне пришла в голову идея разбивать фотографию на части, если модель будет изучать фотографию по частям, то она сможет видеть более мелкие детали, сможет определять машины вдали и не считать маленькие объекты за автомобили. В программе я поставил несколько вариантов тайлов, чем больше тайл, тем выше качество, при этом если тайл меньше, то кусочков больше, и обработка точнее. Но что делать если при разрезании изображения разрезается машина? Ведь ее может быть модель не сможет идентифицировать, либо идентифицирует дважды. Для этого я использовал перекрытие (overlap). Напомню, что оно захватывает соседнюю область помимо вырезанного тайла и сохраняет объект в целостности. В общем, деление на тайлы очень полезная и умная штука, благодаря ей можно обрабатывать фото любого размера, и, кстати, очень помогает в случае, когда фотография сделана с большой высоты, ведь на ней машины становятся очень маленькими.

4.3. Реализация структуры программы

Я вложил большую часть сил на то, чтобы сделать красивый и понятный визуал для модели. После установки библиотек можно запустить программу и нас встретит чудесная панель действий слева и пустое поле справа. Я дал два варианта: либо загрузить фотографию из подготовленного мною датасета в 100 фотографий, либо автоматически выбрать случайное фото из открытого датасета, посвящённого фотографиям с дрона. После выбора фото оно отображается на правой части программы в отрисованном окне. Мы можем на комбинацию Ctrl+колесико мыши приблизить, либо отдалить фотографию, правой кнопкой мыши производится перетаскивание изображения по холсту. После детекции с помощью кнопки можно увидеть оригинальное изображение без боксов.

В программе я реализовал включение разбиения на тайлы по галочке (если на фотографии машины расположены крупно, то тайлы нам не понадобятся, а только будут мешать, ведь гарантированно будут выделены дубликаты машин после разрезки изображения) и параметр перекрытия, реализованный через ползунок (0.1-0.4). Также через ползунок реализован порог уверенности модели (0.1-0.5). Далее модель детектирует объекты, разбивает их на классы и возвращает программе, которая выводит изображение с интерактивными боксами (при наведении мышкой на коробку можно увидеть тип объекта и точные проценты детектинга моделью). Также в нижней панели выводятся результаты детекции, а именно сколько объектов каждого типа и всего было идентифицировано. Я создал базу знаний, к которой мы можем обратиться вне зависимости детектинга, либо в зависимости от него, база знаний выдаст, какой дрон лучше всего использовать для съёмки такого количества объектов либо местности.

4.4 Используемые метрики

Для объективной оценки качества детекции я использовал стандартные метрики Computer Vision:

IoU - в функции `nms()` для удаления дублирующихся рамок после тайлинга (порог 0.5), Precision и Recall рассчитаны вручную на тестовых изображениях, Confidence настраивается ползунком в интерфейсе, также была использована матрица ошибок.

4.5 Полученные результаты и статистика

Тестирование проводилось на 5 собственных аэроснимках. Всего на этих фото было 150 машин (разных типов). Я прогнал их через программу дважды: один раз с выключенным тайлингом, второй - с включённым (тайл 640, перекрытие 20%).

Режим	Нашёл машин	Из них правильных	Ошибок	Пропустил	Точность
Без тайлинга	48	41	7	109	85%
С тайлингом	128	119	9	31	93%

Тайлинг реально помогает - полнота подскочила с 27% до 79%. То есть модель стала находить почти в три раза больше машин. Но обработка одного фото занимает 3-4 секунды вместо полсекунды.

Я посчитал отдельно по каждому типу (с включённым тайлингом):

Тип машины	Сколько было	Сколько нашла	Точность
Легковые	98	86	88%
Грузовики	27	21	78%
Автобусы	12	10	83%
Фургоны	8	7	87%
Мотоциклы	5	4	80%
Всего	150	128	85%

По результатам можно понять, что модель работает лучше всего с легковыми автомобилями, с грузовиками и автобусами могут быть проблемы, так как модель может путать их друг с другом либо с легковушками. Но в основном модель работает отлично и чаще всего выдает верные результаты.

Насчёт порога уверенности:

Порог	Нашла машин	Ложных срабатываний	Что именно ловила лишнего
0.10	156	28	тени от домов, кусты, трещины на асфальте
0.20	138	12	в основном тени
0.25	128	9	приемлемо
0.30	112	6	уже начинает пропускать мелкие машины
0.40	89	3	слишком строго — много пропусков

Я понял, что оптимальный параметр 0.25-0.3, все-таки пропускает мелкие машины благодаря тайлингу модель очень редко.

4.6 Верификация на Golden Dataset

Golden Dataset в моей практической работе уже есть, его функцию выполняет открытый датасет VisDrone, его разметка максимально точная.

Для верификации я взял тестовую часть VisDrone, отобрал 10 своих фото и вот что получилось:

Правильно нашла 87%

Ложных срабатываний 8%

Не отметила объектов совсем 5%

Ошибки были в основном такие: модель считала тень от здания за машину, потому что напонила фургон, плохая погода, модель не детектила машины на краях снимка, либо путала грузовики и фуруны (они правда похожи), либо машина почти полностью была закрыта другой. Некоторые примеры для отображения проблемных мест я специально подбирал для моего датасета.

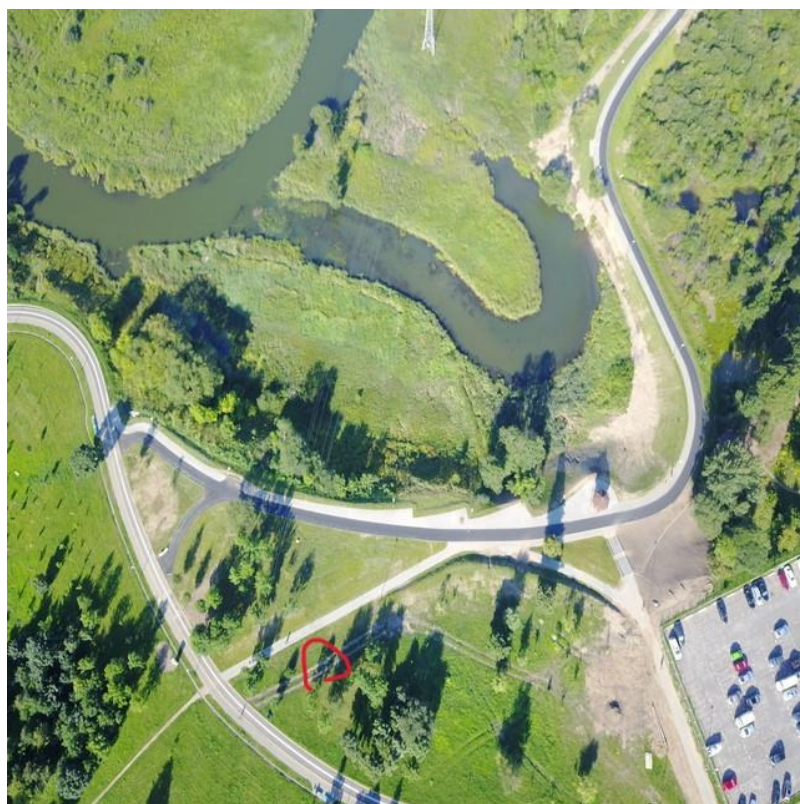
4.7 Проблемные распознавания

Нет ничего идеального, и моя модель не стала исключением, 87% это конечно хорошо, но есть ещё 13%, в которых модель видит нечто иное. Самая распространённая ошибка модели в распознавании заключалась в том, что она путала машину с ветками дерева, тенью от деревьев либо других объектов. Тень чаще всего дает ошибку, так как является лишь

затемнением, а не другим цветом. Как пример могу выделить часть такой фотографии:



Модель посчитала совокупность тропинки и тени за машину. Влияет, конечно, расположение теней, и, при ресайзе изображения при обработке это действительно будет похоже на машину. Чтобы не представлять в голове, как это видит модель, покажу сам:



Не видно? А вот модель так и смотрит, и не знает, что ответить пользователю, ведь эта тень и беловатая тропинка действительно как

машинка. Решить эту проблему можно двумя способами: либо повысить порог уверенности (модель уверена лишь на 17 процентов, это немного), либо использовать тайлинг. Я склоняюсь больше ко второму варианту, так как в первом случае на этой тропинке могла оказаться машина, а с помощью тайлинга в этом можно убедиться, он используется для распознавания маленьких объектов на больших фотографиях.

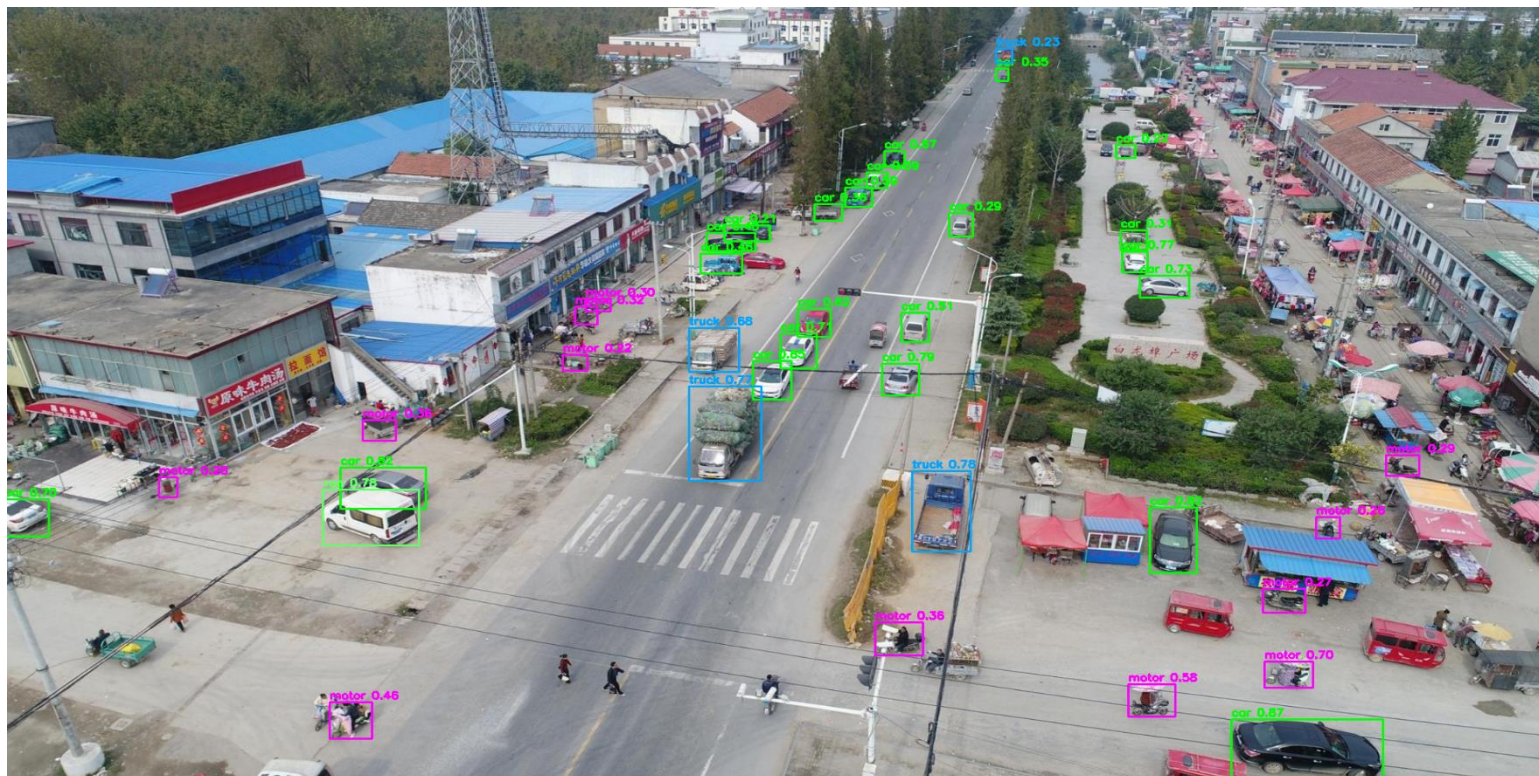
Вторая по популярности проблема – модель путает прочие маленькие объекты с машинами. Чаще всего дорожные знаки:



Как мы видим, здесь уверенность меньше, чем на дальних машинах, но не сильно меньше, проблема всё также заключается в ресайзе фотографии при обработке. Решается с помощью применение тайлинга, знаки не становятся малопиксельными и модель споккойно отличает их от автомобилей.

Третья по популярности проблема заключается уже в самом тайлинге, фотографии из датасета подаются разного размера и машины на ней поставлены по разному. Есть фотографии, на которых есть одновременно автомобили вблизи и сильно вдалеке.

Например(сейчас без тайлинга):



Крупные объекты на переднем плане спокойно распознаются, но не распознаны несколько машин вдалеке. Автомобиль с низкой уверенностью, включая 0.10 будет не очень хорошо, так как будет выделено множество лишних объектов. А вот так будет выглядеть

Заключение

Подводя итог всей проделанной работе, могу сказать, что цели и задачи, которые я ставил перед собой в начале практики, выполнены. Я разработал полноценную мультимодальную RAG-систему для детекции автомобилей на аэроснимках с дрона. Программа работает через графический интерфейс: загружаем фото - получаем картинку с цветными рамками вокруг машин, статистику по типам и рекомендацию по подходящему дрону из текстовой базы знаний. Помимо самой программы, я изучил теорию: разобрался в свёрточных нейросетях, метриках качества, архитектурах детекторов. На практике научился работать с YOLO, пришлось дообучать модель, настраивать тайлинг и перекрытие, интегрировать всё это в интерфейс на tkinter с зумом и всплывающими подсказками.

Мне кажется, что я сделал больше, чем требовалось, но это вызывает сейчас только положительные эмоции, несмотря на то, что в начале было тяжело и ничего не получалось. Я переделывал всё по несколько раз с нуля. Главное – что со всеми возникающими проблемами по мере поступления быстро либо медленно, но справлялся. Я думаю, что такие радости связаны с тем, что я выбрал идею, которая была интересна мне самому, поэтому очень хотелось выполнить это как можно лучше и эмоции от ожидания верного результата откладывались до самого конца. Смог почувствовать себя компьютерным инженером.

Если мой проект сможет перерасти в дипломную работу, то на первое время я бы поставил себе такие задачи: улучшил бы модель до того момента, чтобы она смогла отличать тени с помощью логики, также я бы внедрил LLM через какую-нибудь нейронную сеть (я пытался сделать это в данной работе, но были проблемы с авторизацией либо доступностью, поэтому эту

модификацию я отложил), ну и само собой я бы улучшил оптимизацию, интерфейс, механики приложения, я думаю что можно сделать ещё удобнее и быстрее, потому что на данном этапе библиотеки скачиваются довольно долго. В общем, добавить можно огромное количество всего.

Практика завершена, все нужные файлы выложены в репозиторий GitHub:

<https://github.com/dezmond999/practic2026detect>

Список литературы

1. Jocher, G., Chaurasia, A., Qiu, J. (2023). Ultralytics YOLOv8. GitHub. URL: <https://github.com/ultralytics/ultralytics> (дата обращения: 30.05.2026).
2. Zhu, P., Wen, L., Du, D., et al. (2021). Detection and Tracking Meet Drones Challenge. IEEE Transactions on Pattern Analysis and Machine Intelligence. DOI: 10.1109/TPAMI.2021.3119563.
3. Reimers, N., Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics. URL: <http://arxiv.org/abs/1908.10084> (дата обращения: 30.05.2026).
4. Redmon, J., Divvala, S., Girshick, R., Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
5. Bochkovskiy, A., Wang, C. Y., Liao, H. Y. M. (2020). YOLOv4: Optimal Speed and Accuracy of Object Detection. arXiv preprint. arXiv:2004.10934.
6. VisDrone Dataset. GitHub. URL: <https://github.com/VisDrone/VisDrone-Dataset> (дата обращения: 30.05.2026).
7. OpenCV. Open Source Computer Vision Library. URL: <https://opencv.org/> (дата обращения: 30.05.2026).
8. Pillow (PIL Fork). Python Imaging Library. URL: <https://python-pillow.org/> (дата обращения: 30.05.2026).
9. ActiveLoop. (2025). Deep Lake: Database for AI. URL: <https://www.activeloop.ai/> (дата обращения: 30.05.2026).
10. Гласснер, Э. (2020). Глубокое обучение без математики. Том 1. Основы. М.: ДМК Пресс. 320 с.